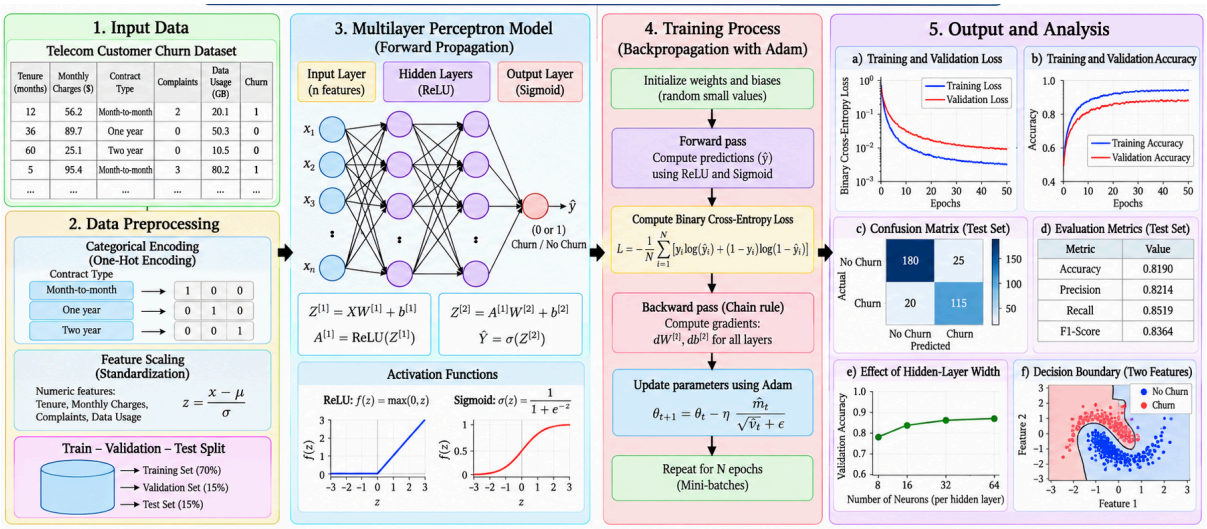
Experiment 9: Multilayer Perceptron (MLP) Implementation

1. Aim

To implement a Multilayer Perceptron (MLP) classifier using TensorFlow and Keras to predict customer churn in a telecommunications company. To preprocess customer data by encoding categorical features and scaling numerical features, train the model using ReLU hidden layers and a sigmoid output layer, evaluate its performance using accuracy, precision, recall, F1-score, and a confusion matrix, and analyze the effects of hidden-layer width on model performance.



2. Mathematical Formulas

1. Forward Propagation

$$\boxed{Z^{[l]} = A^{[l-1]}W^{[l]} + b^{[l]}}$$

$$\boxed{A^{[l]} = f(Z^{[l]})}$$

2. ReLU Activation Function

$$\boxed{\text{ReLU}(z) = \max(0, z)}$$

3. Sigmoid Activation Function

$$\boxed{\sigma(z) = \frac{1}{1 + e^{-z}}}$$

4. Binary Cross-Entropy Loss

$$\boxed{L = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]}$$

5. Accuracy

$$\boxed{\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}}$$

6. Precision

$$\boxed{\text{Precision} = \frac{TP}{TP + FP}}$$

7. Recall

$$\boxed{\text{Recall} = \frac{TP}{TP + FN}}$$

8. F1-Score

$$\boxed{\text{F1} = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}}$$

9. Adam Optimizer

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\boxed{\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}}$$

```python
# Experiment 9: Multilayer Perceptron for Customer Churn

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report,
    ConfusionMatrixDisplay
)

np.random.seed(42)
tf.random.set_seed(42)

# ================================================
# 1. CREATE SAMPLE TELECOM CUSTOMER DATA
# ================================================

n = 1200

tenure = np.random.randint(1, 73, n)
monthly_charges = np.random.uniform(20, 120, n)
contract = np.random.choice(
    ["Month-to-month", "One year", "Two year"],
    n, p=[0.55, 0.25, 0.20]
)
complaints = np.random.poisson(1.0, n)
data_usage = np.random.uniform(1, 100, n)

# Generate churn probabilities from customer features
logit = (
    1.0
    - 0.045 * tenure
    + 0.018 * (monthly_charges - 70)
    + 0.65 * complaints
    - 0.025 * data_usage
    + 1.0 * (contract == "Month-to-month")
    - 0.8 * (contract == "Two year")
)

probability = 1 / (1 + np.exp(-logit))
churn = (np.random.random(n) < probability).astype(int)

df = pd.DataFrame({
    "Tenure": tenure,
    "MonthlyCharges": monthly_charges,
    "Contract": contract,
    "Complaints": complaints,
    "DataUsage": data_usage,
    "Churn": churn
})

print("First five rows of the dataset:")
```

```
print(df.head())
print("\nChurn distribution:")
print(df["Churn"].value_counts())

# ==================================================
# 2. PREPROCESS DATA
# ==================================================

X = df.drop(columns=["Churn"])
y = df["Churn"].values

numeric_features = [
    "Tenure", "MonthlyCharges", "Complaints", "DataUsage"
]
categorical_features = ["Contract"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# Scale numeric features and encode categorical features
preprocessor = ColumnTransformer([
    ("numeric", StandardScaler(), numeric_features),
    ("categorical", OneHotEncoder(handle_unknown="ignore"),
     categorical_features)
])

X_train = preprocessor.fit_transform(X_train).astype("float32")
X_test = preprocessor.transform(X_test).astype("float32")

print("\nTraining samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
print("Number of processed features:", X_train.shape[1])

# Create a validation set from the training data
X_fit, X_val, y_fit, y_val = train_test_split(
    X_train, y_train, test_size=0.20,
    random_state=42, stratify=y_train
)

# ==================================================
# 3. BUILD MLP MODEL
# ==================================================

def build_mlp(width=16):
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(X_fit.shape[1],)),
        tf.keras.layers.Dense(width, activation="relu"),
        tf.keras.layers.Dense(width, activation="relu"),
        tf.keras.layers.Dense(1, activation="sigmoid")
    ])

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )
```

```
    return model

# ================================================
# 4. TRAIN THE MLP
# ================================================

tf.keras.backend.clear_session()
model = build_mlp(width=16)

history = model.fit(
    X_fit, y_fit,
    validation_data=(X_val, y_val),
    epochs=50,
    batch_size=32,
    verbose=0
)

print("\nMLP Training Completed")

# Plot training and validation loss
plt.figure(figsize=(8, 5))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Binary Cross-Entropy Loss")
plt.title("Training and Validation Loss")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

# Plot training and validation accuracy
plt.figure(figsize=(8, 5))
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

# ================================================
# 5. EVALUATE THE MODEL
# ================================================

probabilities = model.predict(X_test, verbose=0).ravel()
predictions = (probabilities >= 0.5).astype(int)

accuracy = accuracy_score(y_test, predictions)
precision = precision_score(y_test, predictions, zero_division=0)
recall = recall_score(y_test, predictions, zero_division=0)
f1 = f1_score(y_test, predictions, zero_division=0)

print("\nEvaluation Metrics")
print("Accuracy :", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall   :", round(recall, 4))
print("F1-score :", round(f1, 4))
```

```
print("\nClassification Report:")
print(classification_report(
    y_test, predictions,
    target_names=["No Churn", "Churn"],
    zero_division=0
))

# Plot confusion matrix
cm = confusion_matrix(y_test, predictions)

ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["No Churn", "Churn"]
).plot(cmap="Blues", values_format="d")

plt.title("Customer Churn Confusion Matrix")
plt.show()

# ==================================================
# 6. COMPARE HIDDEN-LAYER WIDTHS
# ==================================================

widths = [8, 16, 32, 64]
width_results = []

for width in widths:
    tf.keras.backend.clear_session()
    tf.random.set_seed(42)

    candidate = build_mlp(width=width)

    h = candidate.fit(
        X_fit, y_fit,
        validation_data=(X_val, y_val),
        epochs=50,
        batch_size=32,
        verbose=0
    )

    val_prob = candidate.predict(X_val, verbose=0).ravel()
    val_pred = (val_prob >= 0.5).astype(int)

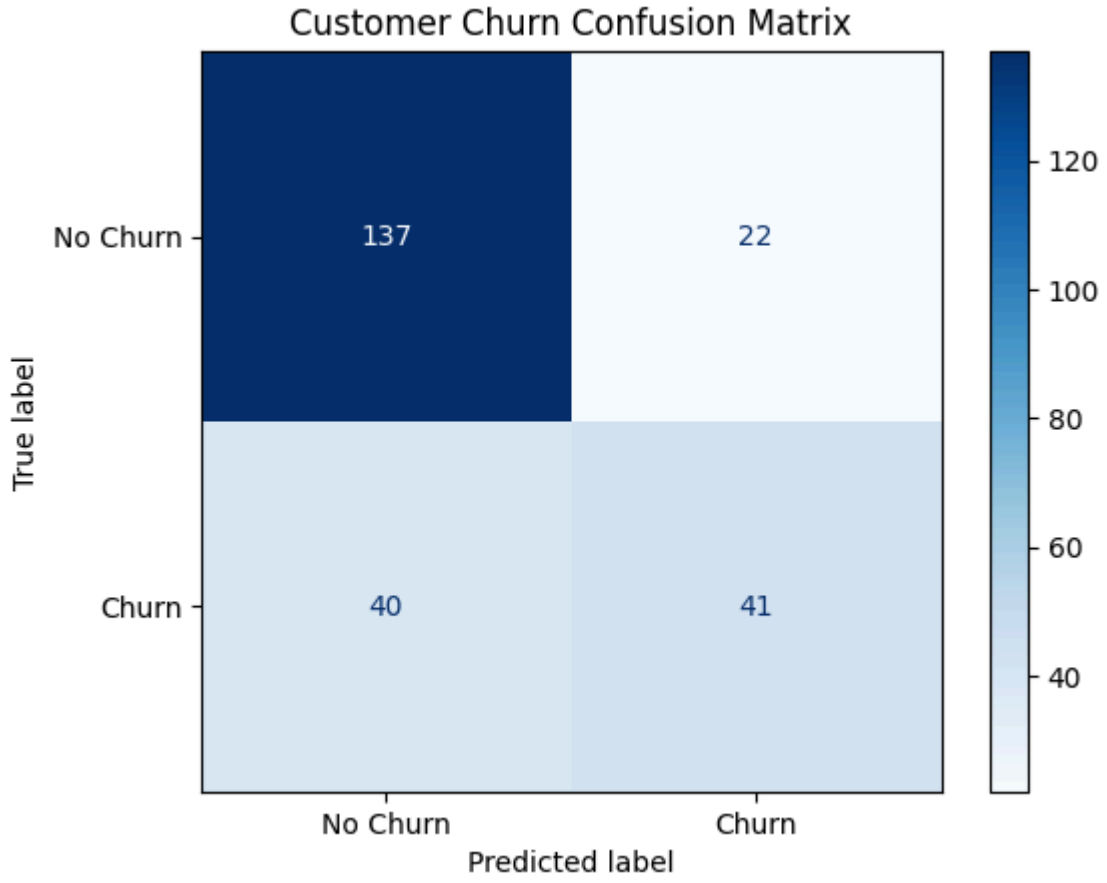
    width_results.append({
        "Hidden Width": width,
        "Final Training Loss": h.history["loss"][-1],
        "Final Validation Loss": h.history["val_loss"][-1],
        "Validation Accuracy": accuracy_score(y_val, val_pred),
        "Validation F1": f1_score(
            y_val, val_pred, zero_division=0
        )
    })

width_df = pd.DataFrame(width_results)

print("\nHidden-Layer Width Comparison:")
print(width_df.round(4).to_string(index=False))
```

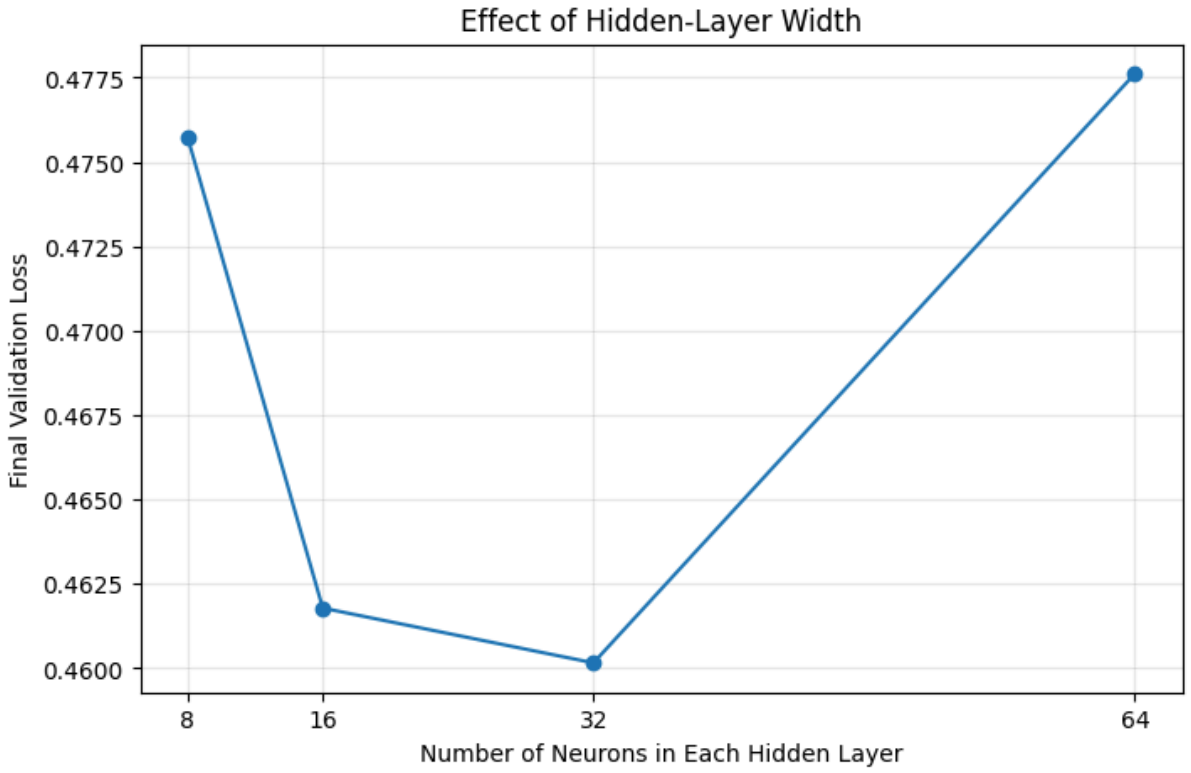
```
# Plot validation loss against width
plt.figure(figsize=(8, 5))
plt.plot(
    width_df["Hidden Width"],
    width_df["Final Validation Loss"],
    marker="o"
)
plt.xlabel("Number of Neurons in Each Hidden Layer")
plt.ylabel("Final Validation Loss")
plt.title("Effect of Hidden-Layer Width")
plt.xticks(widths)
plt.grid(True, alpha=0.3)
plt.show()
```

OUTPUT



Hidden-Layer Width Comparison:

Hidden Width	Final Training Loss	Final Validation Loss	Validation Accuracy	Validation F1
8	0.4797	0.4757	0.7656	0.6565
16	0.4524	0.4618	0.7865	0.6667
32	0.4247	0.4602	0.7760	0.6504
64	0.4030	0.4776	0.7708	



5. RESULT

A Multilayer Perceptron classifier was implemented using TensorFlow and Keras to predict telecom customer churn. Categorical features were encoded, numerical features were standardized, and the network was trained using ReLU hidden layers, a sigmoid output layer, binary cross-entropy loss, and the Adam optimizer. Model performance was evaluated using accuracy, precision, recall, F1-score, and a confusion matrix. Training and validation curves were plotted, and different hidden-layer widths were compared to study their effect on performance.